

Omokai Robotics TakeHome

Challenge 2 Technical Writeup: Autonomous Navigation

Husky UGV · single Intel RealSense RGBD camera · ROS 2 Humble · Nav2 · Gazebo · RViz

*This addresses Challenge 2 of the Omokai takehome: navigating an unknown/partiallyknown environment by reasoning over perceived structure, rather than following a fixed list of waypoints. The codebase is my own past project, built earlier with a friend for a different company's takehome that also scored plant counting reused here per Omokai's explicit allowance to build on past work. Plant counting is **not** part of Challenge 2 and is not graded here; I've kept the description of it in Section 6 only for completeness, since the code is still sitting in the repository.*

0. Summary

A Husky carrying a single, low, forwardfacing RealSense RGBD camera drives itself from a start marker to a goal marker across a fivelane farm field, discovering and covering the traversable lanes as it goes. Lanes and obstacles are found purely from depth geometry a RANSAC ground-plane fit followed by height classification never from colour. Navigation runs on realtime Nav2: a mission node turns what the robot has perceived into a sequence of `NavigateToPose` goals that sweep the field corridor by corridor. There is no fixed list of world waypoints anywhere in the planner. Start and goal poses are read at runtime from the Gazebo markers, not hardcoded. Each run is recorded as an annotated video and a plannedversusactual path overlay.

1. Perception: how depth drives every decision

1.1 What the camera actually gives us

The camera publishes the RGB stream, the depth stream, both cameras' intrinsics, and an organized RGBD point cloud. That's it no LiDAR, no IMU feeding navigation, no GPS, no second camera.

1.2 Lane detection depthgeometric, no colour thresholding

Roughly six times a second, the perception node does the following:

1. **Deproject and transform.** The organized cloud, which arrives in the camera's optical frame, gets downsampled and transformed into `base_footprint` (gravityaligned, zup) using TF. This is the one place RGB and depth genuinely combine: depth supplies the 3D structure, the camera extrinsics from the URDF/TF tree place it in the robot's body frame, and the RGB channel is carried along purely for visualization no perception decision ever looks at colour.

2. **RANSAC groundplane fit.** A RANSAC fit, restricted to nearhorizontal candidate planes, finds the dominant ground plane. This is the actual sceneunderstanding step the challenge is asking for, and it's invariant to how the lanes are angled or coloured it only cares about geometry.
3. **Height classification.** Every point's signed height above that plane decides what it is: within a small tolerance of zero, it's drivable ground; above a threshold, it's an obstacle (the crop beds). Obstacle points become `/farm/obstacles`, which is the only source Nav2's costmap uses.
4. **Lane and bedrow geometry.** In a window ahead of the robot, obstacle points get histogrammed by lateral offset. The peaks in that histogram are bed rows; the midpoints between adjacent rows are lane centres. The nearest corridor becomes a lookahead way-point, and it's what you see drawn as the lane region in the demo video, computed the same way per pixel from depth height.

Why this holds up regardless of orientation, geometry, or appearance. It's invariant to all three by construction, not by tuning. Orientation doesn't matter because the ground comes from a 3D plane fit and beds are found by height above it nothing assumes a lane runs along a fixed image column or world axis, so a skewed bed (one bed in this world is intentionally rotated) is detected exactly the same way as an axisaligned one. Geometry doesn't matter because the corridor centre is always the *local* midpoint between whatever two obstacle rows flank the robot right now, recomputed every frame, so lane spacing and width can vary from lane to lane. Appearance doesn't matter because depth is the only signal used anywhere in the pipeline colour, texture, lighting, and crop appearance never enter a single decision.

2. Navigation: how goals get generated and decisions get made

2.1 Frames and localization

Odometry (`odom` $\rightarrow$ `base_footprint`) comes from the drive plugin's wheel odometry. The `map` $\rightarrow$ `odom` transform is a static, worldaligned identity set to the robot's start pose, so map coordinates equal world coordinates. RTABMap runs as visual SLAM on the single camera for mapping and visualization, but deliberately *not* wired into the navigation TF tree its corrections periodically jump close to a metre on the lowtexture farm floor, and letting that drive the control frame would teleport the path and topple the robot in lanes barely a metre wide. SLAM informs the map; odometry drives the control.

2.2 Start and goal, read at runtime

The start and goal marker poses are queried from Gazebo at launch and published as latched topics. There is no start or goal constant anywhere in the planning code the robot is placed at the start marker as part of simulation setup, the same way it would already be standing on a marked start point in a realworld handout.

2.3 Perceptiondriven coverage

A small state machine turns what's been perceived into Nav2 goals there is no hardcoded trajectory anywhere in it:

INIT $\rightarrow$ COVER $\rightarrow$ SHIFT $\rightarrow$ COVER $\rightarrow$ $\dots$ $\rightarrow$ GO_GOAL $\rightarrow$ DONE

COVER drives the current corridor end to end with a single `NavigateToPose` goal at the far margin; Nav2's planner and controller handle the actual driving on the depthbuilt costmap, tuned to hug the global path so the base stays centred in narrow lanes. **SHIFT** picks the next

corridor as the nearest *perceived* lane centre the midpoint between two detected bed rows rather than a blind fixed step, so the target always lands in an actual gap. A corridor whose perceived bed spacing is too narrow to be traversable is excluded up front; one the planner still can't get through is caught by lack of forward progress and skipped. **GO_GOAL** drives to the exact goal marker once its lane has been covered, retrying on abort since the skidsteer base can't reliably hit an exact final heading. **DONE** publishes a summary: lanes covered, lanes skipped, path length, final goal.

Because the start sits in the southernmost corridor and the goal in a northern one, sweeping south to north naturally covers nearly the whole field *and* ends at the goal, rather than just dashing straight there.

2.4 Why Nav2, not openloop steering

On a base this size, a metrewide lane leaves very little clearance on either side any drift off centre risks the robot clipping a bed. Letting Nav2 and its local controller execute on the perceptionbuilt costmap (which inflates the beds outward) is far more robust than openloop lane following, while still being fully perceptiondriven, since the costmap *is* the perception output.

3. Hardcoded vs. learned full accounting

Learned models: none. No HSV, no fixed RGB thresholds, no colour segmentation, no trained network anywhere in this pipeline. Every decision in perception, navigation, and the (unused, see Section 6) plant counter is classical depth geometry deterministic and auditable. The topic interfaces are modelagnostic, so a learned segmentation or detection head could be dropped in later without touching anything downstream; this submission simply doesn't use one.

The table below lists every fixed quantity that actually matters to Challenge 2 lane detection and navigation along with its value, where it lives, and what kind of constant it is: **G** (a geometric assumption), **T** (a fixed threshold), **M** (a safety margin), **P** (a tuned control/planner parameter), or **S** (simulation setup, not part of the navigation logic itself).

3.1 Lane detection (lane_perception_node.py, geom.py)

#	Parameter	Value	Type	Role
1	ground = dominant nearhorizontal plane		G	the farm floor is the largest planar surface, so it's the RANSAC target
2	RANSAC planenormal gate	$ n_z > 0.85$	G	only accept nearhorizontal candidates, so a bed wall or a plant can't win the fit
3	ground tolerance	0.04 m	T	a point counts as "ground" if within this band of the plane
4	RANSAC budget	40 iters / 0.03 m / 2500 pts	P	iteration count, inlier distance, subsample cap
5	obstacle min height	0.07 m	T	a point this far above ground is an obstacle (beds are ≈ 0.10 m)
6	obstacle max height	0.80 m	T	ceiling above plant foliage, to ignore spurious tall returns
7	obstacles stand above the ground plane		G	beds rise above the floor; the floor itself is free space
8	downsample stride	3	P	keeps the purePython pipeline realtime

#	Parameter	Value	Type	Role
9	forward analysis window	0.454.5 m	T	xrange used for bedrow/lane analysis ahead of the robot
10	lateral analysis window	3.5 m	T	yrange for bedrow analysis
11	bed rows run roughly straight in the window		G	lets a 1D histogram recover rows
12	rowhistogram bin/threshold	0.30 m / 0.10 m / 0.04	T	bin size and peak threshold for bedrow detection
13	lookahead distance	1.8 m	P	how far ahead the lanecentre waypoint is placed
14	robot clearance radius	0.55 m	M	drop obstacle points this close to the base, so it can't trap itself
15	NaN/inf depth pixels dropped		T	removes Gazebo's noreturn pixels before RANSAC

3.2 Coverage mission (farm_navigation/mission_node.py)

#	Parameter	Value	Type	Role
16	default pitch	1.5 m	T	fallback bedrow spacing before enough perception data exists, later refined and clamped
17	min pass spacing	1.2 m	G/T	bed spacing below this is treated as non-traversable, autoskipping the narrow lane
18	default margin	6.0 m	G	drive each lane out to just past the field edge
19	lanematch tolerance	0.6 m	T	how close is "arrived at this lane's latitude"
20	arrival tolerance	0.6 m	T	farend / lateralshift arrival radius
21	impassable progress threshold	1.5 m	T	minimum forward progress before a lane counts as covered rather than skipped
22	navsend grace period	4.0 s	P	real time given to Nav2 to plan and start driving before judging a lane failed this was the fix that stopped premature skips
23	cover retries (max)	2	P	Nav2 recovery retries on a lane before giving up on it
24	perlane timeout	70 s	M	safety backstop per lane
25	goal retries / accept radius	12 / 0.9 m	P	how persistently the final goal is retried, and how close counts as arrived
26	bedrow heading gate	$ \sin(\text{heading}) < 0.30$	G	only fold bedrow detections into the map when reasonably rowaligned
27	bedrow cluster merge	0.40 m	T	merges nearby body detections into one bed centre

3.3 Nav2 / costmaps (farm_navigation/config/nav2_params.yaml)

#	Parameter	Value	Type	Role
28	robot footprint	1.00×0.68 m	M	the actual Husky footprint, used for collision checking

#	Parameter	Value	Type	Role
29	inflation radius	0.36 m	M	just past the inscribed radius, keeps the path off beds without penalizing lane centre
30	cost scaling factor	2.8	P	how sharply cost falls off with distance, tuned so the lane centre stays attractive
31	speed caps	0.24 m/s / 0.8 rad/s	P	kept low for stability in metrewide lanes
32	xy goal tolerance	0.30 m	T	goal position tolerance
33	yaw goal tolerance	3.2 rad	M	deliberately wide open a skidsteer base can't reliably hit an exact heading, and a tight tolerance would abort otherwisesuccessful goals
34	DWB controller critic weights	path/goal/rotation/obstacle weights	P	tuned so the controller hugs the global path, keeping the base centred
35	obstacle layer height/range	0.051.0 m / 6.0 m	T	which cloud points mark the costmap
36	costmap size/resolution	34×24 m and 6×6 m, 0.05 m	P	global and local planning extents and grid resolution
37	global planner	NavFn A*, tol 0.5, allow unknown	P	global planner configuration

3.4 Robot / camera and sim setup

#	Parameter	Value	Type	Role
38	camera mount pose	low, forward, pitched $\approx 15^\circ$ down	S/G	matches a realistic lowmounted forward sensor
39	camera FOV / resolution / clip range	wide FOV, 640×480, 0.1010 m	S	modeled on a D435like depth sensor
40	wheel friction	passive (drive plugin moves the base directly)	S	simulation detail, not part of the navigation logic
41	robot spawn pose	placed at the start marker	S	the planner still reads start/goal at runtime nothing here is baked into the planning code
42	map→odom transform	identity	G	odometry is already worldaligned, so map = odom = world with no spawn pose baked in

Summary: 42 handengineered quantities relevant to Challenge 2, zero learned. Every one of them is a geometric assumption, a fixed threshold, a safety margin, a tuned control parameter, or a simulationsetup detail never a trained weight.

4. Performance against the metrics that matter for this challenge

Numbers below are from a verified headless run (`run_nogpu.sh`, no GPU, software rendering), which is the path actually used to validate this submission end to end.

Metric	Measured result	How the design achieves it
Navigation success	Mission reported complete, goal reached at (5.80, 2.70)	Realtime Nav2 planning/control on the depthbuilt costmap; open yaw tolerance and goalleg retries account for the skidsteer base
Lane coverage	4 lanes covered, 1 skipped, path length 23.2 m	Southtonorth sweep, one corridor at a time, covering up to and including the goal's own lane before the final approach not a straight dash to goal
Path efficiency	Overlay shows the actual trajectory tracking the Nav2 plan closely through the lanes	Nav2's controller is tuned to hug the global path, keeping the base centred

Known limitations and what I'd fix next

Which lane gets skipped isn't perfectly stable under software rendering. The world this was tested against has five lanes total; one is intentionally too narrow to be safely traversable and is meant to be autoskipped. In the verified run, the covered/skipped *count* came out correct (4 covered, 1 skipped), but the specific lane flagged as nontraversable was, by its perceived position, actually the normalwidth lane next to the narrow one not the narrow lane itself, which got driven without issue. I dug into this rather than just accepting the number: the run's log showed depthperceived lane centres carrying up to roughly 0.7 m of offset from the world file's true bed geometry, along with frequent rejected loopclosures from RTABMap throughout the run. Both point to the same cause depthcamera fidelity degrading under CPU/software rendering (there's no GPU on the machine I ran this on), not a bug in the navigation or perception logic itself. Nav2's lifecycle activated cleanly and path length grew continuously with no stuckretry behaviour, which rules out the ABImismatch bug described in the README. On GPUrendered hardware, I'd expect this noise to drop substantially, since the depth camera would be rendering at full fidelity rather than falling back to software rasterization.

Tight lanes are the hardest part for a skidsteer base. With well under half a metre of clearance on either side in the narrower lanes, small centreline errors matter a lot. The 4second grace period before judging a lane failed, plus tuning the controller to hug the global path, make this workable, but a lane is occasionally entered only partially before the mission moves on.

The lane map is necessarily built incrementally. Since the only sensor is a single low, forwardfacing camera, the robot discovers the field as it drives rather than having any global knowledge upfront which is the whole point of Challenge 2, but it does mean the very first lane after the start may be entered before the bed map has enough data to be fully confident. A brief initial survey pass, or a taller camera mount, would densify this map faster.

The natural next step would be to drop a learned segmentation model in behind the same topic interfaces (`/farm/obstacles`, `/farm/lane_center`) that classical geometry currently populates the architecture doesn't need to change, only what fills those topics.

5. Deliverables produced by a run

`output/run_video.mp4` the RGB feed with the detected lane region and navigation status overlaid. `output/path_overlay.png` planned versus actual trajectory, with the covered area and start/goal marked. `/mission/summary` the final report: lanes covered, lanes skipped, path length, goal reached.

6. Appendix plant counting (inherited, not part of this challenge)

The project this was adapted from also included a plantdetection and counting pipeline, built for a different takehome's brief that scored on plant counts specifically. It isolates candidate plants by height above the beds, clusters them, rejects clusters too wide to be a single plant, and de-duplicates detections of the same plant seen from multiple angles by tracking cluster centroids in a fixed world frame. It's still sitting in the `farm_perception` package, but it is **not launched** as part of this submission and isn't relevant to how Challenge 2 is graded it's included here only so the appendix accounts for every piece of code actually present in the repository, not because it's part of the deliverable.